SYSTEM VALIDATION REPORT

Railway Reservation System

1. Introduction

System validation is performed to assess whether the Railway Reservation System supports the passenger workflows and expected user requirements. This report summarizes the validation scenarios documented in the updated test plan, including login, train search, seat availability, booking, payment, PNR confirmation, cancellation/refund, waitlist monitoring and language selection.

The actual results and statuses for the nine user-facing scenarios below are taken from the project's updated Test Plan document. Backend automated tests were also rerun with the MongoDB test database configured; their latest execution results are summarized in this report.

2. Objectives of System Validation

- Validate the passenger-facing workflows against the stated requirements.
- Check that valid inputs are accepted and invalid inputs are handled with appropriate feedback.
- Verify booking/payment states, PNR generation, cancellation and refund behavior.
- Validate waitlist status and supported language selection.
- Summarize test outcomes and identify the scope limitation of database-dependent automated tests.

3. Validation Environment

Application	Railway Reservation System
Architecture / stack	MERN-style application; backend Node.js / Express; MongoDB via Mongoose; frontend React
Backend test runner	Node.js built-in test runner (node --test)
HTTP test utility	Supertest (available in backend development dependencies)
Coverage tool	c8 (installed for coverage reporting)
Test command	npm test (node --env-file=.env --test --test-concurrency=1)
Coverage command	npx c8 --reporter=text --reporter=html npm test
Environment	Windows desktop; backend project directory
Database testing	MONGODB_TEST_URI configured to railway_test; database-dependent automated tests executed

MongoDB test database: configured using MONGODB_TEST_URI=mongodb://127.0.0.1:27017/railway_test. Database-dependent booking lifecycle, payment, PNR/refund, saved language preference and waitlist tests ran in the latest backend test execution.

4. System Validation Scenarios and Test Cases

4.1 User Login

Validation Test ID	SV-01
Scenario / Description	Verify valid login and rejection of invalid credentials.
Preconditions	Registered passenger; login page available.
Test Steps	Login with valid credentials; repeat with incorrect password.
Test Data	Valid: passenger01 / Pass@123; Invalid: passenger01 / Wrong@123.
Expected Result	Valid login grants access; invalid credentials show an error.
Documented Actual Result	Successful login; invalid login rejected with "Invalid email or password."
Status	PASS

4.2 Search Trains

Validation Test ID	SV-02
Scenario / Description	Verify train search with valid journey details and validation of missing destination.
Preconditions	Train search function available; passenger logged in.
Test Steps	Enter Bengaluru as source, Chennai as destination, choose future date, search; repeat with empty destination.
Test Data	Valid: Bengaluru → Chennai on a future date; Invalid: destination empty.
Expected Result	Matching trains appear; missing/invalid data triggers validation and prevents search.
Documented Actual Result	Matching trains appeared; empty destination showed validation message and prevented submission.
Status	PASS

4.3 Seat Availability

Validation Test ID	SV-03
Scenario / Description	Verify availability for a selected train and travel class.
Preconditions	Search completed and at least one train found.
Test Steps	Select train and class; view seat availability; try an unavailable class.
Test Data	Valid: AC 3 Tier; Invalid: unavailable class.
Expected Result	Availability is shown accurately; unavailable class cannot be booked.
Documented Actual Result	For class 3A, 42 of 64 seats were shown available; passenger details form appeared.
Status	PASS

4.4 Ticket Booking

Validation Test ID	SV-04
Scenario / Description	Verify booking with valid passenger details and rejection of invalid details.
Preconditions	Passenger logged in; seats available.
Test Steps	Select train/class; enter passenger details; submit; repeat with invalid details.
Test Data	Valid: Ravi Kumar, age 21, AC 3 Tier; Invalid: empty name, age -2.
Expected Result	Valid details proceed to booking; invalid details are rejected with validation messages.
Documented Actual Result	Booking created; ₹850 UPI payment completed; booking confirmed, PNR generated, details displayed.
Status	PASS

4.5 Online Payment

Validation Test ID	SV-05
Scenario / Description	Verify payment processing and validation of invalid payment details.
Preconditions	Valid booking ready for payment.
Test Steps	Open payment; select UPI; enter details; confirm; repeat with invalid UPI ID.
Test Data	Valid: ₹850, UPI ID valid; Invalid: invalid_format / demo @failbank.
Expected Result	Valid payment succeeds; invalid details/payment are declined and booking remains unconfirmed until success.
Documented Actual Result	Demo failure UPI declined; valid UPI payment completed; status Paid and PNR/payment reference shown.
Status	PASS

4.6 Booking Confirmation & PNR

Validation Test ID	SV-06
Scenario / Description	Verify confirmation and valid PNR display; check handling of non-existing PNR.
Preconditions	Successful booking and payment completed.
Test Steps	Open confirmation; view PNR and booking details; look up a non-existing PNR.
Test Data	Valid: paid booking with generated PNR; Invalid: non-existing PNR.
Expected Result	Confirmation shows generated PNR and correct booking details; unknown PNR gives appropriate message.
Documented Actual Result	PNR lookup displayed confirmed status, train/route/date/class, passenger, amount, method and payment reference.
Status	PASS

4.7 Ticket Cancellation & Refund

Validation Test ID	SV-07
Scenario / Description	Verify cancellation of an eligible confirmed ticket and applicable refund.
Preconditions	Passenger logged in; confirmed ticket eligible for cancellation.
Test Steps	Open booked tickets; select ticket; cancel and confirm; inspect refund status.
Test Data	Valid: confirmed booking, fare ₹850; Invalid: non-existing/already-cancelled booking.
Expected Result	Eligible ticket cancelled and refund shown; invalid or repeated cancellation rejected.
Documented Actual Result	Booking marked Cancelled; ₹765 refunded (90%), ₹85 charge (10%); refund reference displayed.
Status	PASS

4.8 Waitlist Monitoring

Validation Test ID	SV-08
Scenario / Description	Verify current status of a waitlisted booking and handling of unknown PNR.
Preconditions	Passenger logged in with waitlisted booking.
Test Steps	Open waitlist; select booking; view status; try unknown booking.
Test Data	Valid: existing waitlisted booking; Invalid: non-existing PNR.
Expected Result	Current waitlist status shown; unknown PNR returns error/not-found message.
Documented Actual Result	Payment generated PNR; booking remained Waitlisted; user was told automatic confirmation if seats free; Monitor Waitlist option shown.
Status	PASS

4.9 Language Selection

Validation Test ID	SV-09
Scenario / Description	Verify supported language selection and rejection of unsupported language.
Preconditions	Passenger logged in; language settings available.
Test Steps	Open settings; choose supported language; inspect interface; try unsupported language.
Test Data	Valid: supported language; Invalid: unsupported language.
Expected Result	Interface changes to supported language; unsupported option is not applied.
Documented Actual Result	Interface changed from English to Tamil across heading, navigation, booking info and PNR search interface.
Status	PASS

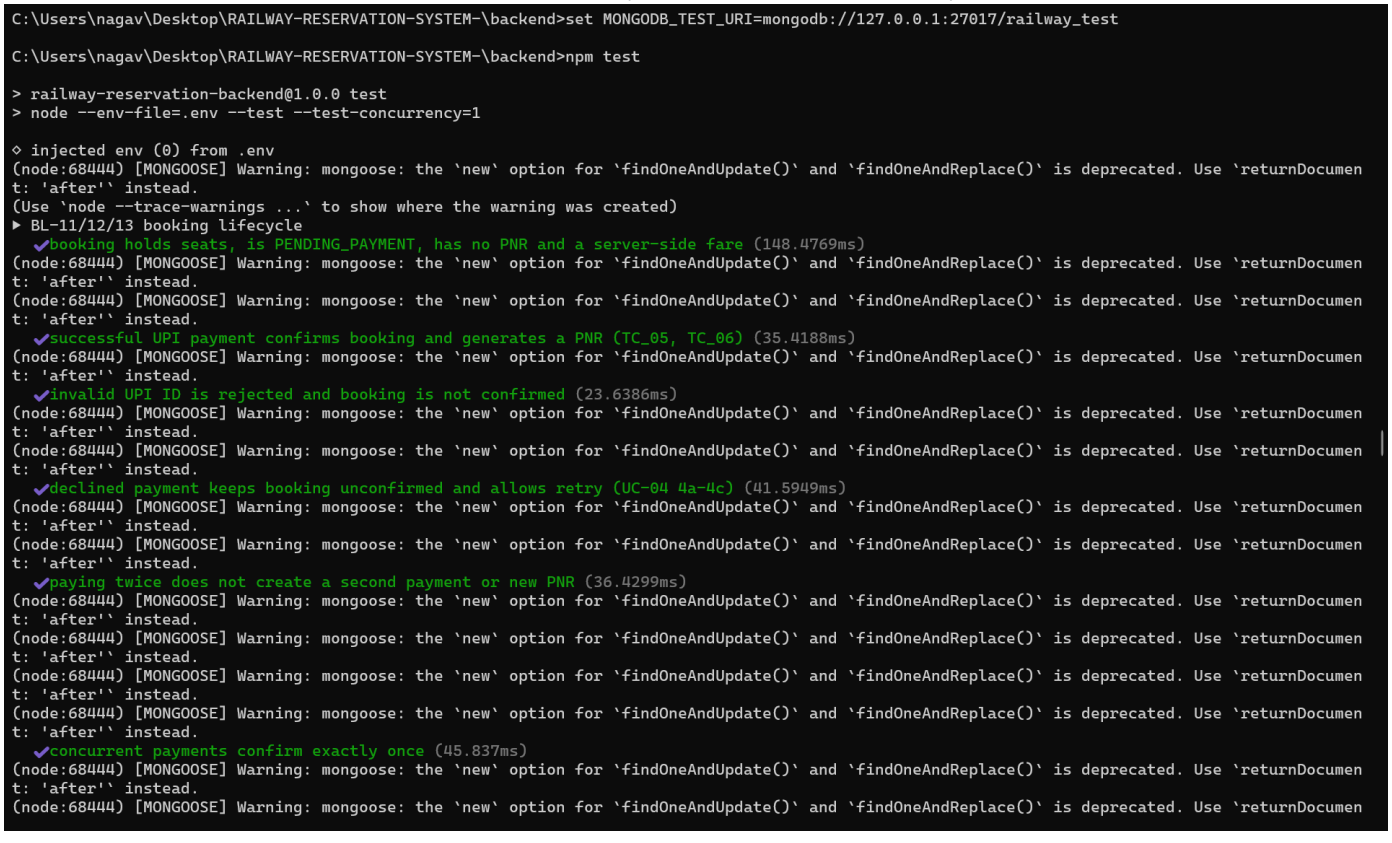
5. Test Execution Summary

Documented validation scenarios	9
Documented scenarios marked PASS	9
Documented scenarios marked FAIL	0
Backend automated tests in reported run	81 passed; 0 failed; 0 skipped (13 suites)
Database integration tests	Executed with MONGODB_TEST_URI configured
Overall documented result	PASS — all 9 documented scenarios marked PASS; automated backend run also passed

The latest backend test run reported 81 passing tests, zero failures and zero skipped tests (13 test suites; approximately 6.89 seconds). The c8 report measured 83.05% statement coverage, 80.00% branch coverage, 100.00% function coverage and 83.05% line coverage. Coverage metrics indicate code exercised by the tests; they do not by themselves establish complete end-to-end system validation.

6. Validation Evidence

1. Screenshot of the successful backend npm test execution (81 passed, 0 failed, 0 skipped).



```
C:\Users\nagav\Desktop\RAILWAY-RESERVATION-SYSTEM-\backend>set MONGODB_TEST_URI=mongodb://127.0.0.1:27017/railway_test

C:\Users\nagav\Desktop\RAILWAY-RESERVATION-SYSTEM-\backend>npm test

> railway-reservation-backend@1.0.0 test
> node --env-file=.env --test --test-concurrency=1

◇ injected env (0) from .env
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
▶ BL-11/12/13 booking lifecycle
  ✔booking holds seats, is PENDING_PAYMENT, has no PNR and a server-side fare (148.4769ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔successful UPI payment confirms booking and generates a PNR (TC_05, TC_06) (35.4188ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔invalid UPI ID is rejected and booking is not confirmed (23.6386ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔declined payment keeps booking unconfirmed and allows retry (UC-04 4a-4c) (41.5949ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔paying twice does not create a second payment or new PNR (36.4299ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
  ✔concurrent payments confirm exactly once (45.837ms)
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:68444) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
```

```
✔BL-15: saved language preference (145.4455ms)
▶ BL-11: payment validation (TC_05)
  ✔accepts a valid UPI ID (0.975ms)
  ✔rejects an invalid UPI ID format (0.2341ms)
  ✔rejects an unsupported payment method (0.093ms)
  ✔accepts a valid card and strips spaces (1.0558ms)
  ✔rejects a card that fails the Luhn check (0.154ms)
  ✔rejects an expired card (0.1724ms)
  ✔rejects a bad CVV (0.166ms)
  ✔luhn and expiry helpers (0.1665ms)
✔BL-11: payment validation (TC_05) (3.9329ms)
▶ BL-11: simulated gateway
  ✔approves a normal UPI payment and masks the reference (0.4946ms)
  ✔declines the demo failure UPI handle (0.2673ms)
  ✔declines the demo failure card and never returns the full number (0.2102ms)
✔BL-11: simulated gateway (1.1244ms)
▶ BL-12: PNR and fares
  ✔generates 10-digit PNRs (1.0703ms)
  ✔rejects malformed PNRs (0.078ms)
  ✔AC 3 Tier fare is Rs 850 per passenger (matches Test Plan) (0.0763ms)
✔BL-12: PNR and fares (1.2978ms)
▶ BL-13: refund rules
  ✔departure is read as IST (1.3716ms)
  ✔90% refund when cancelled 48+ hours before departure (0.1954ms)
  ✔75% refund between 12 and 48 hours (0.1052ms)
  ✔50% refund between 4 and 12 hours (0.0842ms)
  ✔no refund in the last 4 hours, but ticket can still be cancelled (0.1367ms)
  ✔tier boundaries are inclusive of the higher tier (0.0911ms)
  ✔cannot cancel after the train has departed (0.2635ms)
  ✔handles a missing departure time safely (0.1667ms)
  ✔BL-14: a waitlisted ticket is refunded in full at any time before departure (0.257ms)
  ✔BL-14: a waitlisted ticket still cannot be cancelled after the train has departed (0.2852ms)
✔BL-13: refund rules (3.9121ms)
◇ injected env (0) from .env
▶ BL-07: Train Search API
  ✔returns matching trains for a valid Bengaluru to Chennai search (26.6157ms)
  ✔rejects a request with a missing destination (5.7238ms)
  ✔rejects a request with a missing source (3.6845ms)
  ✔rejects identical source and destination (3.7731ms)
  ✔returns zero results for a route without matching sample trains (8.2853ms)
  ✔rejects an invalid journey date (4.4154ms)
  ✔rejects a journey date in the past (3.4688ms)
  ✔search is case-insensitive for station names (3.9563ms)
```

```
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
    ✔a waitlisted ticket cannot be cancelled twice (74.5916ms)
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
    ✔a ticket still waitlisted when the train departs is cancelled with a full refund (68.6282ms)
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
(node:69460) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocumen
t: 'after'` instead.
    ✔confirmed tickets still behave as before: cancel releases the seat (35.1526ms)
  ✔cancelling and expiry (302.0312ms)
✔BL-14 waitlist monitoring (1731.2279ms)
ℹ tests 81
ℹ suites 13
ℹ pass 81
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 6891.2508
```

2. Screenshot of the c8 HTML coverage report summary (83.05% statements, 80.00% branches, 100.00% functions, 83.05% lines).

All files

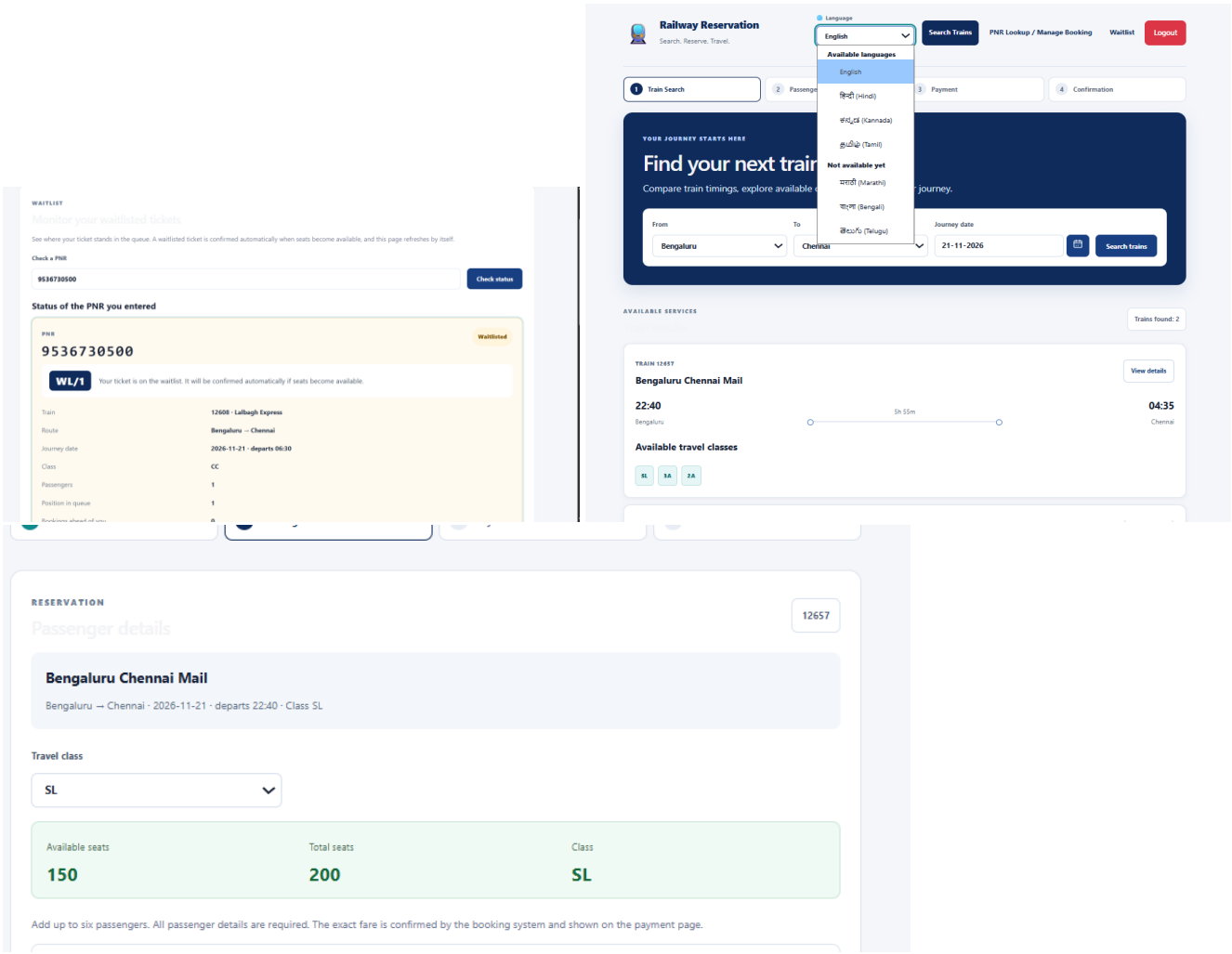
83.05% Statements 1681/2024 **80%** Branches 220/275 **100%** Functions 26/26 **83.05%** Lines 1681/2024

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
src		74.32%	356/479	73.46%	36/49	100%	3/3	74.32%	356/479
src/config		100%	60/60	100%	10/10	100%	3/3	100%	60/60
src/middleware		70.58%	24/34	75%	6/8	100%	1/1	70.58%	24/34
src/models		98.16%	268/273	83.33%	5/6	100%	1/1	98.16%	268/273
src/routes		70.54%	467/662	74.73%	71/95	100%	1/1	70.54%	467/662
src/services		98.06%	506/516	85.98%	92/107	100%	17/17	98.06%	506/516

3. Screenshots of relevant Railway Reservation System screens for login, train search, seat availability, booking/payment, confirmation/PNR, waitlist and language selection, where available.



7. Conclusion

The documented validation scenarios cover the principal passenger workflows of the Railway Reservation System, from authentication and train search through ticket booking, payment, PNR confirmation, cancellation/refund, waitlist monitoring and language selection. All nine scenarios in the updated Test Plan are recorded as PASS. The latest backend automated run completed with 81 passing tests, zero failures and zero skipped tests, with MongoDB test database integration enabled. The measured coverage is 83.05% statements/lines, 80.00% branches and 100.00% functions. Coverage is substantial but not complete;